ТЕХНИЧЕСКОЕ ИССЛЕДОВАНИЕ

Саморазвивающийся поток: архитектура обучающейся системы в `development_system`

*Данное исследование описывает архитектуру саморазвивающегося потока для метациркулярной среды вычислительных потоков на Common Lisp. Предлагается система из четырёх взаимосвязанных потоков — **память**, **обучение**, **диагностика** и **автоматор**, — позволяющих накапливать опыт, распознавать паттерны и автоматически порождать специализированные потоки без перезапуска образа SBCL. Исследование охватывает теоретические основы (in-context learning, episodic memory, self-improvement без fine-tuning), конкретную архитектуру в рамках существующей системы, примеры кода на Common Lisp и пошаговый план внедрения.*

Версия:	1.0
Дата:	14 марта 2026
Проект:	`development_system`
Стек:	Common Lisp (SBCL), OpenRouter, Telegram
Предназначение:	персональная инженерная система обучения

Ключевые слова: Common Lisp, LLM, In-Context Learning, Episodic Memory, Few-Shot, Self-

Improvement, Порождение потоков, GeoDjango, Метациркулярность

Содержание

1 Введение **5**
- 1.1 Постановка задачи . . . 5
- 1.2 Контекст системы . . . 5
- 1.3 Цель исследования . . . 5

2 Теоретические основы **5**
- 2.1 Few-Shot и In-Context Learning . . . 6
- 2.2 Fine-tuning vs In-Context Learning . . . 6
- 2.3 Episodic Memory для агентов . . . 6
  - 2.3.1 MemGPT (2023) . . . 6
  - 2.3.2 RAG over Examples . . . 7
- 2.4 Self-Improvement без обучения модели . . . 7

3 Архитектура системы **7**
- 3.1 Обзор компонентов . . . 7
- 3.2 Поток `память` . . . 8
- 3.3 Поток `обучение` . . . 9
- 3.4 Поток `диагностика` . . . 10
- 3.5 Поток `автоматор` . . . 10

4 Пример end-to-end **11**
- 4.1 Сценарий из жизни GeoDjango разработчика . . . 11
  - 4.1.1 День 1 — обучение . . . 11
  - 4.1.2 День 3 — ещё обучение . . . 11
  - 4.1.3 День 7 — диагностика . . . 11
  - 4.1.4 День 10 — автоматизация . . . 12

5 Реализация **12**
- 5.1 Поток `память` (скелет) . . . 12
- 5.2 Поток `диагностика` (ключевая логика) . . . 14
- 5.3 Поток `автоматор` (триггер и порождение) . . . 14

6 Эволюция и обратная связь **15**
- 6.1 Жизненный цикл . . . 15
- 6.2 Механизм обратной связи . . . 15

6.3 Защита от деградации 16

7 Roadmap **16**

7.1 День 1 — Минимальный MVP 16

7.2 Неделя 1 — Полный цикл имитации 16

7.3 Неделя 2 — Автоматизация 17

7.4 Месяц 1 — Production 17

7.5 Долгосрочно 17

8 Риски и ограничения **18**

8.1 Стоимость LLM-запросов 18

9 Заключение **18**

9.1 Ключевые выводы 18

9.2 Критерий успеха 19

9.3 Следующие шаги 19

1. Введение

1.1. Постановка задачи

Инженер, работающий ежедневно в области Python/Django/ГИС, накапливает паттерны решения задач: оптимизация N+1 запросов в GeoDjango, обработка геометрий, работа с PostGIS. Большинство этих решений типовые, однако каждый раз требуют умственных усилий на воспроизведение.

Идея: создать систему, которая проходит три фазы:

1. **Запоминание** — накопление пар (проблема → решение) из реальной практики
2. **Имитация** — распознавание похожих проблем и воспроизведение решений
3. **Автоматизация** — при высокой уверенности самостоятельное порождение специализированного потока

1.2. Контекст системы

`development_system` — метациркулярная среда:

Компонент	Роль
Мастер (SBCL)	Telegram-бот, polling loop, хранение истории
Поток	Файл `*.lisp` с пакетом `:поток-ИМЯ` и функцией `выполнить`
Порождатель	Метапоток: по описанию генерирует и загружает новый поток

Ключевое свойство — **без перезапуска**: новый поток компилируется через `compile-file + load` в живой образ. Слой генерации кода совпадает со слоем исполнения — классическая метациркулярность.

1.3. Цель исследования

Спроектировать архитектуру саморазвивающегося потока, органично встраивающегося в существующую систему и обеспечивающего постепенный переход от имитации к автоматизации с явными контрольными точками.

2. Теоретические основы

2.1. Few-Shot и In-Context Learning

In-context learning (ICL) — способность LLM решать задачи на основе примеров в промпте без изменения весов модели:

```
Пример
 1: проблема<> -> решение<>Пример
 2: проблема<> -> решение<>
...Новаяпроблема
 : <???> -> ?
```

Listing 1: Структура ICL-промпта

Ключевые свойства ICL:

- Работает без обучения — используем готовую модель
- Качество растёт с количеством релевантных примеров (оптимум: 5–15)
- Порядок примеров важен: ближайшие к запросу — последними
- Чем конкретнее формат примеров, тем стабильнее вывод

2.2. Fine-tuning vs In-Context Learning

Характеристика	Fine-tuning	ICL
Изменение весов	Да	Нет
Стоимость адаптации	Высокая (обучение)	Низкая (токены)
Скорость адаптации	Часы / дни	Мгновенно
Catastrophic forgetting	Да	Нет
Гибкость	Низкая	Высокая

Вывод

Для персональной обучающейся системы **fine-tuning избыточен**. ICL обеспечивает достаточное качество при мгновенной адаптации и нулевой инфраструктуре обучения.

2.3. Episodic Memory для агентов

Episodic memory — память о конкретных эпизодах (событиях), в отличие от semantic memory (общие знания). Две ключевые работы:

2.3.1. MemGPT (2023)

LLM как операционная система с иерархической памятью:

- **Main context** — рабочая память (окно контекста)
- **Archival memory** — долговременное хранилище с поиском
- **Recall memory** — поиск по ключевым словам

В нашей системе: `поток-память` = archival memory с LLM-поиском.

2.3.2. RAG over Examples

Retrieval-Augmented Generation применительно к примерам:

1. Кодировать проблему в embedding
2. Найти k ближайших проблем в хранилище
3. Включить примеры в промпт
4. Сгенерировать решение

Проблема: нужен embedding-сервис. Альтернатива — LLM-сравнение (дороже по токенам, но проще в реализации; достаточно для MVP).

2.4. Self-Improvement без обучения модели

Что НЕ применимо

RLHF и DPO требуют доступа к весам модели и обучающей инфраструктуры. Для персональной системы это избыточно.

Что применимо — Constitutional AI (частично): модель сама оценивает свои ответы по набору принципов. В нашей системе: `поток-диагностика` выполняет self-critique — оценивает сгенерированное решение перед выдачей пользователю.

Prompt-based Self-Improvement (практический подход):

1. Генерируем решение
2. Просим LLM оценить уверенность (0–100)
3. Если уверенность $>$ порога — применяем автоматически
4. Иначе — запрашиваем подтверждение

3. Архитектура системы

3.1. Обзор компонентов

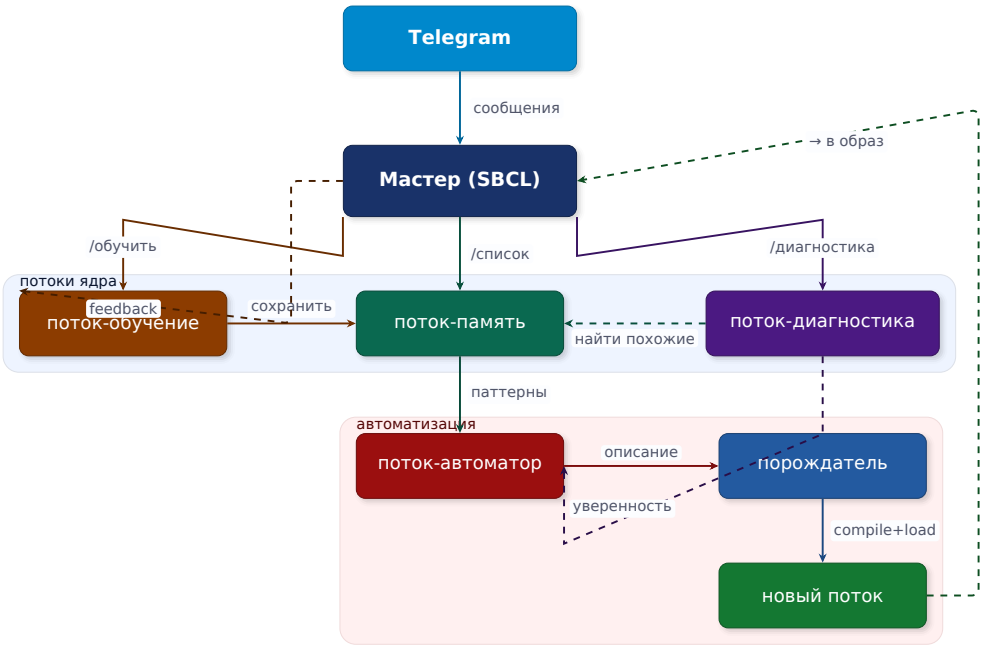

Рис. 1: Архитектура саморазвивающегося потока

3.2. Поток память

Назначение: долговременное хранилище пар (проблема → решение) с поиском по сходству.

Интерфейс:

```
запустить
/ памятьдобавить : проблема=N+1 в GeoDjango | решение=select_related
    запустить
/ памятьнайти : медленный queryset с Pointзапустить
/ памятьсписокзапустить
/ памятьудалить : 3
```

Формат хранения — JSON:

```jsonc
{
  "версия": 1,
  "примеры": [
    {
      "id": 1,
      "проблема": "N+1 запросыв  GeoDjango приобходесвязанныхобъектов   ",
      "решение": "select_related для FK, prefetch_related для M2M",
      "теги": ["django", "geodjango", "n+1"],
```

```
      "дата": "2026-03-14",
      "оценка": 1,
      "применений": 3,
      "паттерн": "orm-n+1"
    }
  ],
  "паттерны": {
    "orm-n+1": {
      "название": "N+1 оптимизацияв  Django ORM",
      "случаев": 5,
      "успешность": 1.0,
      "автоматизирован": true,
      "поток": "оптимизатор-orm"
    }
  }
}
```

Listing 2: Структура файла памяти

Три стратегии поиска (по возрастанию качества/стоимости):

#	Стратегия	Качество	Стоимость
1	Keyword matching	Низкое	Бесплатно
2	LLM-сравнение	Среднее	Токены
3	Embedding + cosine	Высокое	API + модель

Рекомендация для MVP: стратегия 2 (LLM-сравнение) — оптимальный баланс простоты и качества.

3.3. Поток обучение

Назначение: принимает новую пару от пользователя, валидирует, сохраняет в память, анализирует паттерн.

Входной формат:

```
проблема
: N+1 запросовприобходе   Route.objects.all() сдоступомк   waypointsрешение
: Route.objects.prefetch_related('waypoints')теги
: django, n+1, prefetch
```

Логика обработки:

1. Парсинг входа (проблема, решение, теги)
2. Проверка на дубликаты через `поток-память`
3. Если похожая проблема уже есть — предложить обновить или добавить как вариант
4. Сохранение
5. Анализ паттерна: «Это новый паттерн или вариация существующего?»

3.4. Поток `диагностика`

Назначение: при новой проблеме ищет похожие, оценивает применимость, предлагает решение с уверенностью.

Логика генерации решения:

```
Найденыпохожиепроблемыирешения
    :

1. Проблема: похожая< 1>Решение
   : решение< 1>

2. Проблема: похожая< 2>Решение
   : решение< 2>Новаяпроблема

 : текст<>Предложирешение

 . Посленегоукажи  :РЕШЕНИЕ
: ...УВЕРЕННОСТЬ
: <0–100>ОБОСНОВАНИЕ
: ...
```

Listing 3: Структура промпта диагностики

Интерпретация уверенности:

Диапазон	Статус	Действие
85–100%	Высокая	Можно автоматизировать
70–84%	Хорошая	Рекомендуется попробовать
50–69%	Средняя	Проверить внимательно
0–49%	Низкая	Только гипотеза

3.5. Поток автоматор

Назначение: мониторинг паттернов и порождение специализированных потоков при достижении порогов.

Пороги автоматизации:

```
(defparameter пороги**предложение
  '((:    . 30)     ; минимумдлявыводапредложенияприменение
    (:     . 70)     ; автоматическоеприменениесподтверждениемавтоматизация
    (: . 85)))  ; порождениеновогопотока
```

Listing 4: Параметры порогов в Common Lisp

Триггеры автоматизации:

- Уверенность $\geq 85\%$
- Паттерн встречался ≥ 3 раз
- Пользователь подтверждал решение ≥ 2 раз

4. Пример end-to-end

4.1. Сценарий из жизни GeoDjango разработчика

4.1.1. День 1 — обучение

```
> обучить/проблема
: N+1 запросовприобходе   Region.objects.all()сдоступомк
             region.cityрешение
: Region.objects.select_related('city') -- один JOINтеги
: django, n+1, select_related

< Сохранено (id=1). Новыйпаттерн .
```

4.1.2. День 3 — ещё обучение

```
> обучить/проблема
: Медленнаязагрузка  Point с prefetch связанных Polygonрешение
: Point.objects.prefetch_related('polygons').only('id','geom')теги
: geodjango, prefetch

< Сохранено (id=2). Частичноесходствос   id=1.Формируетсяпаттерн
   : Оптимизация" ORM запросов"
```

4.1.3. День 7 — диагностика

```
> диагностика/ Route.objects.all() собходом  route.waypoints.all()

< Найдено 2 похожихпроблемы :

  1. (90%) N+1 приобходе  Region с city
  2. (75%) Медленнаязагрузка  Point с prefetch PolygonРЕШЕНИЕ

  : Route.objects.prefetch_related('waypoints')УВЕРЕННОСТЬ
  : 87%
  -- классический N+1, prefetch загрузитвсе  waypoints
     однимзапросомПрименить

  ? данет[/]

> да

< Успешно. Паттерн "orm-n+1": 3 случая, 100% успешность.
    Рассматриваюавтоматизацию
   ...
```

4.1.4. День 10 — автоматизация

```
< Обнаруженустойчивыйпаттерн  : "N+1 оптимизацияв  Django ORM"Случаев
  : 5, успешность: 100%, средняяуверенность : 89%
     Предлагаюсоздатьпотокоптимизатор

     «-orm».Создать

  ? данет[/]

> да

< Потокоптимизатор «-orm» порождёнизагруженвобраз    .запустить
  / оптимизатор-orm описание" медленного queryset"
```

Результат

За 10 дней система прошла путь от нуля до специализированного автоматического потока, требующего минимального участия человека.

5. Реализация

5.1. Поток память (скелет)

```
(defpackage потокпамять:-
  (:use :cl)
  (:export выполнить#: добавитьпример#:-найтипохожие
           #:- получитьвсе#:-удалитьпример
           #:- обновитьоценку#:-))

(in-package потокпамять:-)

(defparameter путьпамяти*-*
  (merge-pathnames "память.json" (user-homedir-pathname)))

;;; LLMсравнение- двухпроблем
(defun похожи-р проблема(-1 проблема-2)
  "Возвращает да:, частично: илинет :"
  (let* промпт(( (format nil "Проблема А: ~АПроблема~% Б: ~А~%~Похожи
                              ? Ответь: да, частично, нет"проблема
                         -1 проблема-2))ответ
         ( (string-downcase (llmзапрос- промпт))))
    (cond ((search "да" ответ) да      :)
          ((search "частично" ответ) частично:)
          (t нет                        :))))

;;; Поиск k ближайших
(defun найтипохожие- проблема( &key (k 3))
  (let* соценками((-
          (mapcar (lambda п()
                    (cons похожи(-р проблема
                                  (cdr (assoc проблема: п)))п
                          ))примеры
                  ()))отсортированы
         (
          (sort соценками-
                (lambda (a b)
                  (> (case (car a) да(: 2) частично(: 1) (t 0))
                     (case (car b) да(: 2) частично(: 1) (t 0)))))))
    (remove-if (lambda (x) (eq (car x) нет:))
```

```
                (subseq отсортированы 0 (min k (length отсортированы))))))

;;; Точкавходапотока
(defun выполнитьзадача ()
  (handler-case
      (let команда(( парситькоманду(- задача)))
        (case (first команда)добавить
          (: (format nil "Сохранено (id=~A)"добавитьпример
                              (- (second команда)
                                                (third команда))))найти
          (: форматироватьрезультаты   (-найтипохожие
                      (- (second команда))))список
          (: форматироватьсписок  (- получитьвсе(-)))удалить
          (: удалитьпример (- (second команда)))))
    (error (e) (format nil "Ошибка: ~A" e))))
```

Listing 5: поток-память.lisp --- ключевые функции

5.2. Поток диагностика (ключевая логика)

```
(defun диагностикаскритикой-- проблема()
  "Генерирует решениеикритическиегооценивает    ."
  (let* похожие(( потокпамятьнайтипохожие(-:- проблема :k 5))первичное
         ( сгенерироватьрешение(- проблемапохожие ))критика
         (
          (llmзапрос-
           (format nil "Решение: ~AПроблема~%: ~A~%~Критическиоцени
                        :~%~
                       1. Точнолирешаетпроблему   ?~%~
                       2. Побочныеэффекты ?~%~
                       3. Нужныуточнения ?"первичноепроблема
                   ))))
    (format nil "~AСамопроверка~%~%:~%~A"первичноекритика
             )))

(defun извлечьуверенность- ответ()
  "Парсит УВЕРЕННОСТЬ: <N> изответа  LLM."
  (let поз(( (search "УВЕРЕННОСТЬ:" (string-upcase ответ))))
    (if поз
        (parse-integer
         (remove-if-not #'digit-char-p
                       (subseq ответ (+ поз 12) (+ поз 15))))
```

```
        50)))
```

Listing 6: Двухэтапная диагностика с self-critique

5.3. Поток автоматор (триггер и порождение)

```
(defparameter минслучаев*-*    3)
(defparameter минуспешность*-* 0.8)

(defun готовкавтоматизации---р статистика()
  (and (>= (getf статистикаслучаев :) минслучаев   *-*)
       (>= (getf статистикауспешность :) минуспешность*-*)
       (>= (getf статистикаприменений :)
           (* 2 (getf статистикаслучаев :)))))

(defun породитьспециализированный- статистика()
  (let описание(( (format nil
    "Создай поток ~Аоптимизатор-.~%~Задача
     : принятьописаниепроблемыивернутьрешение     .~%~
        Паттернынроблемарешение
      ->:~%~{- ~A -> ~A~%~}"
    (getf статистиканазвание :)
    (mapcan (lambda п()
              (list (cdr (assoc проблема: п))
                    (cdr (assoc решение: п))))
            (subseq (getf статистикапримеры :) 0 5)))))
               потокпорождательвыполнить
    (-: описание)))
```

Listing 7: Анализ готовности паттерна к автоматизации

6. Эволюция и обратная связь

6.1. Жизненный цикл

Этап	Примеров	Поведение системы
Накопление	< 3	Сохраняет примеры, не предлагает решений
Имитация	$3 - 10$	Few-shot промпт, решения с уверенностью
Обобщение	$10+$	Выделяет абстрактные паттерны
Автоматизация	≥ 3 + критерии	Порождает специализированный поток

6.2. Механизм обратной связи

```
хорошо
/ 5         -- пример id=5 помог, +1 коценкеплохо
/ 5 не" то"  -- id=5 неподошёл , -1 + сохранитькакантипаттерн
```

Антипаттерн сохраняется с префиксом «НЕ ДЕЛАТЬ:» и влияет на future-диагностику — система учится избегать неправильных решений.

6.3. Защита от деградации

1. **Decay** — влияние примеров без применений за 30+ дней постепенно снижается
2. **Порог применений** — минимум 2 успешных применения до участия в автоматизации
3. **Self-critique** — LLM оценивает паттерн перед порождением потока
4. **Карантин** — новые потоки первые 5 вызовов работают в режиме «предложение», не «применение»

7. Roadmap

7.1. День 1 — Минимальный MVP

✓ `поток-память.lisp` с keyword matching (без LLM)

✓ Добавление примеров, поиск, список

✓ Тест: 3 примера + поиск похожего

```
(defun найтипословам-- проблема( примеры)
  (let слова(( (uiop:split-string (string-downcase проблема))))
    (sort примеры #'>
          :key (lambda п()
                 (count-if (lambda c()
                             (search c (string-downcase
                                        (cdr (assoc проблема: п)))))слова
                           )))))
```

Listing 8: Поиск по словам без LLM для быстрого старта

7.2. Неделя 1 — Полный цикл имитации

- LLM-поиск по сходству в `поток-память`
- `поток-обучение` с валидацией дубликатов
- `поток-диагностика` с уверенностью
- Feedback (`/хорошо`, `/плохо`)

7.3. Неделя 2 — Автоматизация

- `поток-автоматор` с порогами
- Интеграция с `порождатель`
- Карантин для новых потоков
- Тест полного цикла end-to-end

7.4. Месяц 1 — Production

- Self-critique, decay устаревших примеров
- UI для просмотра и редактирования памяти
- Экспорт / импорт, статистика

7.5. Долгосрочно

- Embedding-based поиск (локальная модель или API)
- Иерархия паттернов (мета-паттерны)
- Интеграция с IDE через LSP extension

8. Риски и ограничения

Риск	Описание	Митигация
Контекстное окно	При 100+ примерах невозможно включить все в промпт	Жёсткий лимит: топ-5; batch-сравнение
Drift качества	Устаревшие паттерны деградируют	Decay, тегирование версий
Overfitting	Система учится неоптимальному стилю	Self-critique + флаг «проверено»
Безопасность	Порождённый код может быть опасен	Sandboxing, timeout, карантин
Стоимость LLM	N запросов на каждый поиск	Batch, кэш, дешёвая модель для сравнения

8.1. Стоимость LLM-запросов

При 100 примерах и batch-сравнении: 100×5 токенов ≈ 500 токенов $\approx \$0.001$ за один поиск. Приемлемо для персонального использования.

9. Заключение

9.1. Ключевые выводы

1. **In-context learning достаточен.** Fine-tuning избыточен и создаёт больше проблем (catastrophic forgetting, инфраструктура), чем решает.
2. **Метациркулярность — главное преимущество.** Порождатель уже существует в системе; саморазвивающийся поток — его естественное расширение.
3. **Gradual autonomy.** Переход от имитации к автоматизации должен быть постепенным, с явными checkpoint'ами: feedback, пороги, карантин.
4. **Human-in-the-loop критичен.** Без обратной связи система неизбежно деградирует. Feedback — не опция, а архитектурное требование.

9.2. Критерий успеха

Через месяц система считается успешной, если:

- Инженер тратит меньше времени на повторяющиеся проблемы
- Есть ≥ 1 автоматически порождённого полезного потока
- Точность предложений $> 70\%$

9.3. Следующие шаги

1. Скопировать скелеты потоков (раздел 5) в `мастер/потоки/`
2. Добавить 5 реальных примеров из GeoDjango-практики
3. Использовать, давать feedback, наблюдать за эволюцией

Документ подготовлен для Артемия. Версия 1.0, 14 марта 2026. Проект: `development_system`.